МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Факультет информационных технологий и прикладной математики
Кафедра вычислительной математики и программирования

Лабораторная работа №3 по курсу «Информационный поиск»

Токенизация

Студент: К. А. Арусланов
Группа: М8О-403Б-23
Преподаватель: А. А. Кухтичев
Дата: 21.10.2026
Оценка:

Москва, 2026

Содержание

1 Постановка задачи 3

2 Метод решения 3

2.1 Корпус и подготовка данных ... 3

2.2 Алгоритм и правила ... 3

2.3 План тестирования и измерений ... 5

3 Результаты 5

3.1 Количество и длина токенов ... 5

3.2 Производительность ... 6

3.3 Качество выделения и ограничения ... 7

3.4 Автотесты и воспроизведение ... 8

4 Вывод 9

1 Постановка задачи

Нужно реализовать процесс разбиения текстов документов на токены, который потом будет использоваться при индексации. Для этого потребуется выработать правила, по которым текст делится на токены. Необходимо описать их в отчёте, указать достоинства и недостатки выбранного метода. Привести примеры токенов, которые были выделены неудачно, объяснить, как можно было бы поправить правила, чтобы исправить найденные проблемы.

В результатах выполнения работы нужно указать следующие статистические данные:

- количество токенов;
- среднюю длину токена.

Кроме того, нужно привести время выполнения программы, указать зависимость времени от объёма входных данных. Указать скорость токенизации в расчёте на килобайт входного текста. Является ли эта скорость оптимальной? Как её можно ускорить?

2 Метод решения

2.1 Корпус и подготовка данных

Темой корпуса являются русскоязычные и англоязычные публикации о компьютерных играх восьми источников. Входные данные — публикации, сохранённые роботом второй работы в коллекции `documents` базы MongoDB. Используются только пригодные для индексации записи без ошибки парсера. Из каждой записи экспортируются заголовок и основной текст. Публикации без основного текста участвуют заголовком; страницы-каталоги исключаются до токенизации.

Python-экспортёр создаёт неизменяемый локальный снимок с бинарным файлом `corpus.bin`, таблицей соответствия ID и URL `documents.jsonl` и описанием `manifest.json`. Для снимка сохраняется SHA-256. В бинарном формате поля предваряются длинами в байтах, поэтому переводы строк и нулевые символы не нарушают границы документов. Заголовок и основной текст остаются отдельными полями. Сырой HTML при этом не читается.

Экспорт идёт курсором MongoDB, обработка — по одному документу. Весь корпус не требуется помещать в оперативную память. Снимок позволяет повторять измерения на одинаковых данных и исключить MongoDB из замеров производительности токенизатора. Экспорт не меняет базу. В описании снимка зафиксированы начало и конец экспорта. Обычный курсор не создаёт транзакционный снимок MongoDB: при одновременной обкачке состав базы может меняться. Все приведённые подсчёты относятся к завершённому локальному файлу.

2.2 Алгоритм и правила

Основная программа написана на C++17 с использованием разрешённой для токенизации STL. Алгоритм самостоятельно декодирует UTF-8 и последовательно просматривает кодовые точки. Версия правил обозначена `gaming-v3`.

– Буквы ASCII, русские буквы, латиница Latin-1 и Extended-A, а также ASCII-цифры входят в состав слов. Цифры сохраняются и внутри названий: `PS5` превращается в `ps5`, `3D` — в `3d`.

– Регистр нормализуется по явным таблицам поддерживаемых букв, независимо от локали системы. Буква «ё» не заменяется на «е».

– Внутренние дефисы и апострофы сохраняются. Типографские апострофы приводятся к ASCII-апострофу, обычный и неразрывный дефисы — к `-`. Соединитель допустим только внутри слова перед буквой или цифрой. Среднее и длинное тире считаются разделителями.

– Обозначения `C++`, C# и их варианты с номером стандарта, например `C++17`, сохраняются целиком. Правило применяется к отдельному `C`/`c`, а не к любой последовательности букв перед плюсом.

– В числовой последовательности точка сохраняется перед следующей цифрой: `3.0.1` становится одним токеном. Поддерживается префикс `v`, например `V1.2.3` превращается в `v1.2.3`. Конечная точка предложения отделяется. Это лексическое правило: десятичные числа и IP-адреса также сохраняются целиком, без определения типа сущности.

– Цепочка из трёх и более одиночных букв через точки превращается в один токен без точек: `S.T.A.L.K.E.R.` и `STALKER` дают одинаковый терм `stalker`. Конечная точка необязательна. Две буквы через точку не склеиваются, многобуквенные сегменты не распознаются как такая цепочка. Правило действует на поддерживаемую латиницу и русские буквы, без словаря названий игр.

– Пробелы, переводы строк, остальная пунктуация и emoji разделяют слова. Стоп-слова, короткие слова и числа не удаляются.

– Комбинируемые знаки U+0300–U+036F присоединяются к начатому токену. Unicode-нормализация NFC/NFKC не выполняется. Остальные письменности пока считаются разделителями; полное Unicode case folding не реализовано.

– Повреждённый UTF-8 и обрезанный бинарный формат считаются ошибкой входных данных. Программа завершается с ненулевым кодом возврата.

Каждый токен содержит ID документа, имя поля, позицию с нуля внутри поля, начальное и конечное байтовые смещения и нормализованный терм. Смещения относятся к исходному тексту, а не к строке после нормализации. Такое представление можно использовать при построении позиционного индекса и сниппетов. Тот же модуль подходит для обработки поисковых запросов. Лемматизация и стемминг в этой работе не выполняются.

Просмотр занимает линейное время относительно числа байтов текста. В памяти остаются текущий документ и буфер токена; глобальный словарь терминов не строится. Длина токена считается в кодовых точках нормализованной строки, включая внутренние соединители, а не в байтах UTF-8. Это также не число видимых графем: буква и отдельный комбинируемый знак дают две кодовые точки. Средняя длина равна отношению суммы длин токенов к их количеству; при отсутствии токенов принимается равной нулю.

2.3 План тестирования и измерений

Модульные тесты на C++ с GoogleTest вызывают токенизатор напрямую и проверяют русский и английский текст, регистр и «ё», цифры, типографские соединители, обозначения языков программирования, версии, цепочки одиночных букв через точки, диакритику, границы UTF-8, позиции внутри полей, пустые и большие тексты. Тесты обработчика командной строки проверяют вывод TSV и JSON-статистику. Отдельно проверяются повреждённая кодировка, обрезанные записи, недопустимые длины, неверные аргументы, сохранение существующих файлов и отсутствие опубликованного частичного результата после ошибки. CTest также запускает сам исполняемый файл для проверки справки и обработки неизвестного режима. Тесты экспортёра проверяют отбор документов, сохранение публикаций с одним заголовком и отсутствие изменений в базе. Тесты демонстрации проверяют соответствие фрагмента исходной статье, байтовые границы, выбор документа и вывод UTF-8.

Для измерения зависимости времени от размера берутся каждый восьмой, четвёртый, второй документ снимка и весь снимок. Подвыборки распределены по исходному порядку URL, поэтому включают разные сайты. Тексты не дублируются для искусственного увеличения объёма. На каждой подвыборке проводится один прогрев и 5 измерений; основным результатом служит медиана. Все повторы сохраняются в `benchmark.json`.

Отдельно измеряется сканирование с нормализацией и счётчиками, а также полное чтение и обработка бинарного файла. В этих повторах запись токенов выключена. Ещё один запуск сохраняет полный TSV и показывает стоимость вывода. Создание процесса и экспорт MongoDB в приведённое внутреннее время не входят. Файловый кеш прогрет; результаты не характеризуют чтение с холодного диска. Везде используется десятичный килобайт: 1 КБ = 1000 байт.

3 Результаты

3.1 Количество и длина токенов

Эксперимент выполнен 21 октября 2026 года на снимке из 12 497 публикаций, из них 121 содержат только заголовок. Суммарный размер заголовков и основного текста — 40 324,69 КБ. Статистика рассчитывается по обоим полям без удаления стоп-слов и словоформ.

Показатель	Значение
Количество документов	12 497
Количество токенов	4 140 679
Средняя длина токена, кодовых точек	5,308

Источник	Документы	Токены	Средняя длина
Eurogamer	1058	521 285	4,682
GameMAG	2442	482 025	5,700
GamingOnLinux	378	121 603	4,734
Игромания	2411	731 247	5,734
IXBT.games	1817	338 056	5,880
PC Gamer	1477	888 458	4,625
PlayGround	1290	215 523	5,858
StopGame	1624	842 482	5,537

Статистика получена по всем пригодным записям завершённого экспорта. Объём пока не достигает требуемых курсом 30–50 тысяч документов. После продолжения обкачки необходимо повторить экспорт и тот же эксперимент. Копии текстов для увеличения числа документов не добавлялись.

3.2 Производительность

Программа собрана с оптимизацией `-O3` и стандартом C++17 на Windows AMD64. Точная версия компилятора, его флаги и SHA-256 бинарного файла сохранены вместе с результатами. Объём в таблице включает только UTF-8 заголовки и тексты, без служебных полей бинарного формата.

Документы	Текст, КБ	Обработка, мс	С чтением, мс	Скорость, КБ/с
1563	5 147,28	51,415	57,767	89 104,9
3125	9 682,72	100,796	114,002	84 934,6
6249	19 961,70	199,960	224,238	89 020,3
12497	40 324,69	389,638	431,545	93 442,7

На всём снимке медиана времени обработки составляет 389,638 мс, с чтением — 431,545 мс. Скорость чтения и обработки равна 93 442,7 КБ/с, или 0,0107 мс/КБ. Запуск с сохранением полного TSV занимает 1 581,327 мс: форматирование и запись результата добавляют работу, которой нет в измерениях без вывода. По внешнему таймеру медиана полного запуска без TSV составляет 441,188 мс, а отдельный запуск с TSV — 1 594,473 мс. Эти значения дополнительно включают создание процесса и передачу его небольшого JSON-ответа вызывающему Python-скрипту.

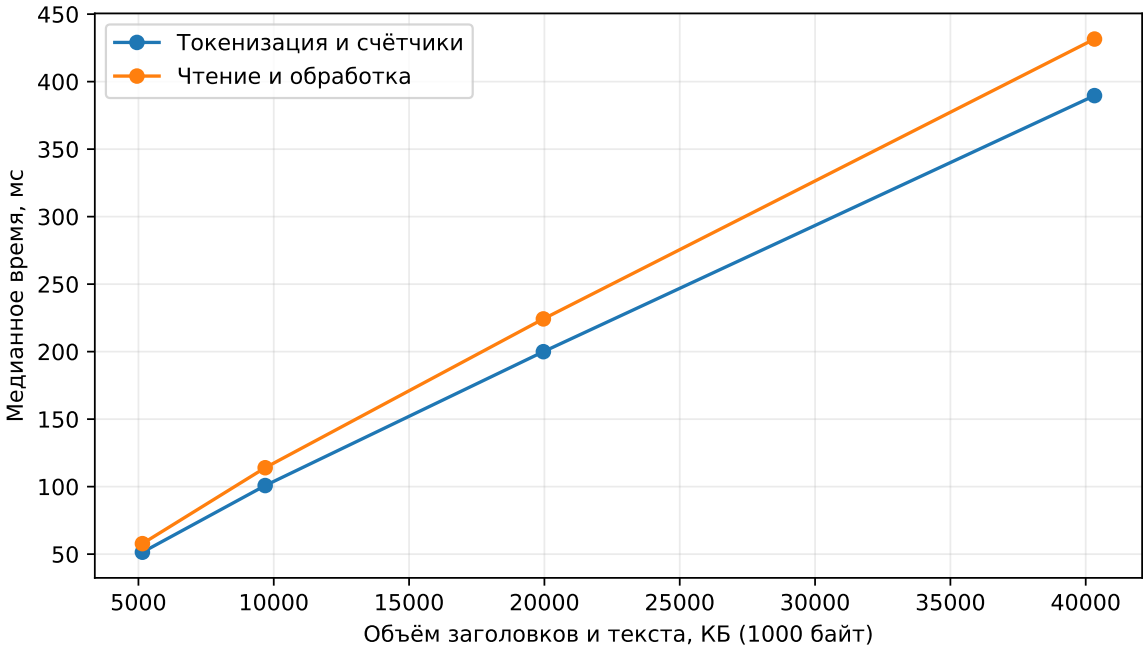


Рис. 1 — Зависимость медианного времени от объёма входного текста.

Линейный проход соответствует ожидаемой зависимости порядка $O(B)$, где B — объём текста. На небольших подвыборках также влияют накладные расходы, состав русских и английских текстов и планирование процессов. Четырёх измеряемых объёмов недостаточно для вывода о скорости на миллионном корпусе.

По порядку сложности полный просмотр текста улучшить нельзя: нужно прочитать входные символы. При этом измеренная реализация не доказана оптимальной по постоянным затратам. Возможные ускорения — отдельный быстрый путь для ASCII, уменьшение расходов вызова обработчика токенов и пакетная запись более компактного результата. Параллельная обработка независимых документов также возможна; сравнивать её следует отдельным экспериментом с теми же правилами и проверкой одинакового результата.

3.3 Качество выделения и ограничения

В сохранённых публикациях найдены следующие характерные случаи. Короткие фрагменты и ожидаемый результат включены в тесты GoogleTest.

Фрагмент	Результат	Наблюдение и возможное улучшение
`C++`, C# (PC Gamer)	`c++`, `c#`	В первой версии оба обозначения превращались в `c`. Добавлено правило сохранения специальных суффиксов; теперь три названия различаются.
`3.0.1` (GameMAG)	`3.0.1`	В первой версии получались три числа. Теперь точка между цифрами сохраняется, а конечная точка предложения отделяется.
`S.T.A.L.K.E.R.`	`stalker`	Цепочка одиночных букв теперь распознаётся целиком. Точки удаляются, поэтому написание `STALKER` даёт тот же терм.
`Half-Life` (GameMAG)	`half-life`	Название сохраняется, но запрос `Half Life` даёт два токена. Для сопоставления вариантов понадобится согласованное правило альтернативного представления.
`Pokémon` (IXBT.games)	`pokémon`	Диакритика сохраняется; `Pokemon` будет другим термом. Возможен отдельный вариант без диакритики, но его следует оценить на неоднозначных названиях.

Пример с C++ и C# взят из публикации PC Gamer, версия — из новости GameMAG о Destruction AllStars. Положительные примеры — новость GameMAG о Black Mesa и материал IXBT.games о патенте Nintendo.

Название `S.T.A.L.K.E.R.` встречается и в сохранённой публикации GameMAG о КРИ Awards. Его нормализация демонстрируется на исходном фрагменте этой статьи. Обратная сторона общего правила — `a.b.c` также даст `abc`, даже если это инициалы или части доменного имени. Для различения таких случаев понадобились бы дополнительные ограничения или контекст. Другие ограничения — неполный набор письменностей и отсутствие NFC. Новые правила нужно применять одинаково к документам и запросам, меняя версию токенизатора.

3.4 Автотесты и воспроизведение

Число успешно выполненных автоматических проверок 22 октября 2026 года составило 43. Из них 34 запускаются через CTest: 17 тестов правил токенизации, 9 тестов обработчика командной строки и 6 тестов демонстрации написаны на C++ с GoogleTest; ещё 2 проверки запускают сам исполняемый файл. На Python остаются 9 проверок вспомогательных компонентов: 4 теста экспортёра и 5 тестов совместимости прежней демонстрации. Алгоритм токенизации в Python-тестах не дублируется.

Часть тестов содержит несколько входных случаев. Полный журнал приложен как `doc/lab3/tests.txt`; данные эксперимента находятся в `statistics.json`, `benchmark.json` и `benchmark.csv` того же каталога. Для тестов подготовлен отдельный Docker-образ с компилятором, CMake и GoogleTest. После однократной сборки образа проверки работают без сети и MongoDB. Исходники подключаются только для чтения, результаты сборки сохраняются в отдельном томе:

```
docker compose --profile tests build tokenizer-tests
docker compose run --rm --no-deps tokenizer-tests
```

Полный эксперимент (сборка, оба набора проверок, экспорт и замеры) запускается одной командой:

```
python run_lab3.py --config crawler.large.yaml \
  --output data/tokenization/run_001 --report-dir doc/lab3
```

В примере перенос строки дан для оболочки Bash; команду также можно ввести одной строкой. Требования и отдельные команды описаны в `doc/lab3_run.md`. Снимки корпуса и полные токены хранятся локально в `data/tokenization/` и не включаются в Git.

Демонстрация также написана на C++ и использует тот же модуль токенизации напрямую. После сборки её можно запустить командой:

```
docker compose run --rm --no-deps tokenizer-tests tokenizer_demo
```

Она выбирает реальные статьи из последнего локального снимка и выводит источник, ID, заголовок, URL, исходный фрагмент и таблицу токенов. Колонка `original` извлекается из исходного текста по байтовым границам, возвращённым C++-программой. Позиции и смещения относятся к целому полю статьи, а не к началу показанного фрагмента. Демонстрируются версии, обозначения языков и нормализация названий с точками. Произвольный документ можно выбрать опцией `-document-id`, фрагмент — опцией `-find`.

Исходный код и отчёт предназначены для ветки `lab3`: https://github.com/kirbasss/MAI-IR/tree/lab3.



Рис. 2 — QR-код для перехода к ветке `lab3` репозитория.

4 Вывод

В этой работе я реализовал токенизацию игровых публикаций и проверил её на русских и английских текстах. Было интересно увидеть, что даже простое разбиение на слова требует нескольких решений: сохранять ли дефис, как обрабатывать апострофы и что делать с цифрами в названиях игр. Особенно наглядными оказались C++ и C#: сначала после удаления пунктуации они превращались в одинаковый токен. Я уточнил правила, и теперь эти обозначения, как и номера версий игр, сохраняются целиком. Для названий вроде `S.T.A.L.K.E.R.` я добавил объединение одиночных букв через точки: теперь их можно сопоставлять с обычным написанием без точек.

Я получил статистику токенов, измерил скорость обработки и подготовил повторяемый запуск экспериментов и демонстрацию на настоящих статьях. Позиции и исходные границы слов сохранены, поэтому результат можно использовать в следующих работах. Дальше нужно расширить реально скачанный корпус, повторить измерения и отдельно добавить обработку словоформ, сохранив возможность точного сопоставления.